# DATA RETRIEVE

In [6]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import plotly.express as px
```

In [7]:
```
import seaborn as sns
```

In [8]:
```
data = pd.read_csv(r"C:\Users\Lenovo\Downloads\Food_Delivery_Times (2).csv")
```

In [9]:
```
data
```

Out[9]:

| | Order_ID | Distance_km | Weather | Traffic_Level | Time_of_Day | Vehicle_Type | Preparation_Time_min | Courier_Experience_yrs |
|---|---|---|---|---|---|---|---|---|
| **0** | 522 | 7.93 | Windy | Low | Afternoon | Scooter | 12 | 1.0 |
| **1** | 738 | 16.42 | Clear | Medium | Evening | Bike | 20 | 2.0 |
| **2** | 741 | 9.52 | Foggy | Low | Night | Scooter | 28 | 1.0 |
| **3** | 661 | 7.44 | Rainy | Medium | Afternoon | Scooter | 5 | 1.0 |
| **4** | 412 | 19.03 | Clear | Low | Morning | Bike | 16 | 5.0 |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... |
| **995** | 107 | 8.50 | Clear | High | Evening | Car | 13 | 3.0 |
| **996** | 271 | 16.28 | Rainy | Low | Morning | Scooter | 8 | 9.0 |
| **997** | 861 | 15.62 | Snowy | High | Evening | Scooter | 26 | 2.0 |
| **998** | 436 | 14.17 | Clear | Low | Afternoon | Bike | 8 | 0.0 |
| **999** | 103 | 6.63 | Foggy | Low | Night | Scooter | 24 | 3.0 |

1000 rows × 9 columns

In [10]:
```
data.head(10)
```

Out[10]:

| | Order_ID | Distance_km | Weather | Traffic_Level | Time_of_Day | Vehicle_Type | Preparation_Time_min | Courier_Experience_yrs | De |
|---|---|---|---|---|---|---|---|---|---|
| **0** | 522 | 7.93 | Windy | Low | Afternoon | Scooter | 12 | 1.0 | |
| **1** | 738 | 16.42 | Clear | Medium | Evening | Bike | 20 | 2.0 | |
| **2** | 741 | 9.52 | Foggy | Low | Night | Scooter | 28 | 1.0 | |
| **3** | 661 | 7.44 | Rainy | Medium | Afternoon | Scooter | 5 | 1.0 | |
| **4** | 412 | 19.03 | Clear | Low | Morning | Bike | 16 | 5.0 | |
| **5** | 679 | 19.40 | Clear | Low | Evening | Scooter | 8 | 9.0 | |
| **6** | 627 | 9.52 | Clear | Low | NaN | Bike | 12 | 1.0 | |
| **7** | 514 | 17.39 | Clear | Medium | Evening | Scooter | 5 | 6.0 | |
| **8** | 860 | 1.78 | Snowy | Low | Evening | Car | 20 | 6.0 | |
| **9** | 137 | 10.62 | Foggy | Low | Evening | Scooter | 29 | 1.0 | |

In [11]:
```
data.tail(10)
```

Out[11]:

| | Order_ID | Distance_km | Weather | Traffic_Level | Time_of_Day | Vehicle_Type | Preparation_Time_min | Courier_Experience_yrs |
|---|---|---|---|---|---|---|---|---|
| **990** | 122 | 17.97 | Windy | Medium | Morning | Scooter | 28 | 7.0 |
| **991** | 615 | 17.53 | Clear | Low | Afternoon | Bike | 14 | 6.0 |
| **992** | 21 | 12.43 | Rainy | High | Morning | Bike | 13 | 6.0 |
| **993** | 701 | 10.89 | Windy | Medium | Evening | Bike | 7 | 0.0 |
| **994** | 72 | 4.37 | Clear | Medium | Evening | Scooter | 6 | 7.0 |
| **995** | 107 | 8.50 | Clear | High | Evening | Car | 13 | 3.0 |
| **996** | 271 | 16.28 | Rainy | Low | Morning | Scooter | 8 | 9.0 |
| **997** | 861 | 15.62 | Snowy | High | Evening | Scooter | 26 | 2.0 |
| **998** | 436 | 14.17 | Clear | Low | Afternoon | Bike | 8 | 0.0 |
| **999** | 103 | 6.63 | Foggy | Low | Night | Scooter | 24 | 3.0 |

In [12]:
```
data.columns
```

Out[12]:
```
Index(['Order_ID', 'Distance_km', 'Weather', 'Traffic_Level', 'Time_of_Day',
       'Vehicle_Type', 'Preparation_Time_min', 'Courier_Experience_yrs',
       'Delivery_Time_min'],
      dtype='object')
```

# Exploratory Data Analysis (EDA)

UNIQUE

In [13]:
```
data['Traffic_Level'].nunique()
```

Out[13]: 3

In [14]:
```
data['Weather'].nunique()
```

Out[14]: 5

DESCRIBE

In [15]:
```
data.describe(include ='all')
```

Out[15]:

| | Order_ID | Distance_km | Weather | Traffic_Level | Time_of_Day | Vehicle_Type | Preparation_Time_min | Courier_Experience |
|---|---|---|---|---|---|---|---|---|
| **count** | 1000.000000 | 1000.000000 | 970 | 970 | 970 | 1000 | 1000.000000 | 970.000 |
| **unique** | NaN | NaN | 5 | 3 | 4 | 3 | NaN | I |
| **top** | NaN | NaN | Clear | Medium | Morning | Bike | NaN | I |
| **freq** | NaN | NaN | 470 | 390 | 308 | 503 | NaN | I |
| **mean** | 500.500000 | 10.059970 | NaN | NaN | NaN | NaN | 16.982000 | 4.579 |
| **std** | 288.819436 | 5.696656 | NaN | NaN | NaN | NaN | 7.204553 | 2.914 |
| **min** | 1.000000 | 0.590000 | NaN | NaN | NaN | NaN | 5.000000 | 0.000 |
| **25%** | 250.750000 | 5.105000 | NaN | NaN | NaN | NaN | 11.000000 | 2.000 |
| **50%** | 500.500000 | 10.190000 | NaN | NaN | NaN | NaN | 17.000000 | 5.000 |
| **75%** | 750.250000 | 15.017500 | NaN | NaN | NaN | NaN | 23.000000 | 7.000 |
| **max** | 1000.000000 | 19.990000 | NaN | NaN | NaN | NaN | 29.000000 | 9.000 |

# DATA CLEANSING - Checking NULL Values

In [16]:
```
data.isnull().sum()
```

Out[16]:
```
Order_ID                  0
Distance_km               0
Weather                  30
Traffic_Level            30
Time_of_Day              30
Vehicle_Type              0
Preparation_Time_min      0
Courier_Experience_yrs   30
Delivery_Time_min         0
dtype: int64
```

In [17]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 9 columns):
 #   Column                  Non-Null Count  Dtype  
---  ------                  --------------  -----  
 0   Order_ID                1000 non-null   int64  
 1   Distance_km             1000 non-null   float64
 2   Weather                 970 non-null    object 
 3   Traffic_Level           970 non-null    object 
 4   Time_of_Day             970 non-null    object 
 5   Vehicle_Type            1000 non-null   object 
 6   Preparation_Time_min    1000 non-null   int64  
 7   Courier_Experience_yrs  970 non-null    float64
 8   Delivery_Time_min       1000 non-null   int64  
dtypes: float64(2), int64(3), object(4)
memory usage: 70.4+ KB
```

## DUPLICACY CHECKING

In [18]:
```
data.duplicated()
```

Out[18]:
```
0      False
1      False
2      False
3      False
4      False
       ...
995    False
996    False
997    False
998    False
999    False
Length: 1000, dtype: bool
```

DROPPING NULL VALUES

In [20]:
```
#data.fillna(data.mean(), inplace = True)
```

USING MEAN() TO FILL NULL VALUES WILL ONLY BE APPLICABLE TO NUMERCIAL COLUMNS CARRYING NULL VALUES. FOR CATEGORICAL COLUMNS, WE HAVE TO USE MODE.

In [21]:
```
data.info()
#it shows all the null values of numercial columns
#are filled up with the mean values of the respective columns.
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 9 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Order_ID                1000 non-null   int64
 1   Distance_km             1000 non-null   float64
 2   Weather                 970 non-null    object
 3   Traffic_Level           970 non-null    object
 4   Time_of_Day             970 non-null    object
 5   Vehicle_Type            1000 non-null   object
 6   Preparation_Time_min    1000 non-null   int64
 7   Courier_Experience_yrs  970 non-null    float64
 8   Delivery_Time_min       1000 non-null   int64
dtypes: float64(2), int64(3), object(4)
memory usage: 70.4+ KB
```

REMOVING NULL VALUES FOR THE CATEGORICAL COLUMNS USING MODE()

In [22]:
```
data.columns
```

Out[22]:
```
Index(['Order_ID', 'Distance_km', 'Weather', 'Traffic_Level', 'Time_of_Day',
       'Vehicle_Type', 'Preparation_Time_min', 'Courier_Experience_yrs',
       'Delivery_Time_min'],
      dtype='object')
```

In [23]:
```
for column in data.columns:
    if data[column].dtype =='object' or data[column].dtype.name =='category':
        data[column].fillna(data[column].mode()[0], inplace = True)
        #df[column].mode()[0]: Accesses the first (and usually only) mode of the column
    else:
        data[column].fillna(data[column].mean(), inplace = True)
```

```
C:\Users\Lenovo\AppData\Local\Temp\ipykernel_18564\2256455101.py:6: FutureWarning: A value is trying to be set on a c
opy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which
we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or d
f[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  data[column].fillna(data[column].mean(), inplace = True)
C:\Users\Lenovo\AppData\Local\Temp\ipykernel_18564\2256455101.py:3: FutureWarning: A value is trying to be set on a c
opy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which
we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or d
f[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  data[column].fillna(data[column].mode()[0], inplace = True)
```

WE ARE USING IF/ELSE INSIDE A FOR LOOP, WHERE FOR THE CATEGORICAL COLUMNS IT WILL EXECUTE MODE ELSE, IT WILL EXECUTE MEAN.

In [24]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 9 columns):
 #   Column                  Non-Null Count  Dtype  
---  ------                  --------------  -----  
 0   Order_ID                1000 non-null   int64  
 1   Distance_km             1000 non-null   float64
 2   Weather                 1000 non-null   object 
 3   Traffic_Level           1000 non-null   object 
 4   Time_of_Day             1000 non-null   object 
 5   Vehicle_Type            1000 non-null   object 
 6   Preparation_Time_min    1000 non-null   int64  
 7   Courier_Experience_yrs  1000 non-null   float64
 8   Delivery_Time_min       1000 non-null   int64  
dtypes: float64(2), int64(3), object(4)
memory usage: 70.4+ KB
```

NUMBER OF UNIQUE IN EACH COLUMN IN DATASET

In [25]:
```
data.nunique()
```

Out[25]:
```
Order_ID                1000
Distance_km              785
Weather                    5
Traffic_Level              3
Time_of_Day                4
Vehicle_Type               3
Preparation_Time_min      25
Courier_Experience_yrs    11
Delivery_Time_min        108
dtype: int64
```

COUNTING TOTAL VALUES UNDER EACH UNIQUE POINT

In [26]:
```
data['Courier_Experience_yrs'].value_counts(ascending = False)
```

Out[26]:
```
Courier_Experience_yrs
6.000000    109
9.000000    108
1.000000    107
8.000000    101
2.000000     99
4.000000     94
7.000000     91
0.000000     91
5.000000     90
3.000000     80
4.579381     30
Name: count, dtype: int64
```

In [27]:
```
data['Preparation_Time_min'].value_counts(ascending = False)
```

Out[27]:
```
Preparation_Time_min
14    52
16    49
25    48
10    47
9     46
11    46
17    45
6     44
22    42
8     42
27    40
28    40
26    40
29    39
21    39
20    39
12    38
23    38
7     37
24    37
19    35
13    30
5     30
15    29
18    28
Name: count, dtype: int64
```

In [28]:
```
data.columns
```

Out[28]:
```
Index(['Order_ID', 'Distance_km', 'Weather', 'Traffic_Level', 'Time_of_Day',
       'Vehicle_Type', 'Preparation_Time_min', 'Courier_Experience_yrs',
       'Delivery_Time_min'],
      dtype='object')
```

In [29]:
```
category =['Weather','Traffic_Level', 'Time_of_Day','Vehicle_Type', 'Preparation_Time_min', 'Courier_Experience_yrs']

for column in category:
    print(data[column].value_counts(ascending = False))
```

```
Weather
Clear    500
Rainy    204
Foggy    103
Snowy     97
Windy     96
Name: count, dtype: int64
Traffic_Level
Medium    420
Low       383
High      197
Name: count, dtype: int64
Time_of_Day
Morning      338
Evening      293
Afternoon    284
Night         85
Name: count, dtype: int64
Vehicle_Type
Bike       503
Scooter    302
Car        195
Name: count, dtype: int64
Preparation_Time_min
14    52
16    49
25    48
10    47
9     46
11    46
17    45
6     44
22    42
8     42
27    40
28    40
26    40
29    39
21    39
20    39
12    38
23    38
```

```
7     37
24    37
19    35
13    30
5     30
15    29
18    28
Name: count, dtype: int64
Courier_Experience_yrs
6.000000    109
9.000000    108
1.000000    107
8.000000    101
2.000000     99
4.000000     94
7.000000     91
0.000000     91
5.000000     90
3.000000     80
4.579381     30
Name: count, dtype: int64
```

GROUPBY

In [30]:
```
data.groupby('Traffic_Level').count()
```

Out[30]:

| Traffic_Level | Order_ID | Distance_km | Weather | Time_of_Day | Vehicle_Type | Preparation_Time_min | Courier_Experience_yrs | Delive |
|---|---|---|---|---|---|---|---|---|
| **High** | 197 | 197 | 197 | 197 | 197 | 197 | 197 | |
| **Low** | 383 | 383 | 383 | 383 | 383 | 383 | 383 | |
| **Medium** | 420 | 420 | 420 | 420 | 420 | 420 | 420 | |

In [31]:
```
df = data.groupby('Traffic_Level')['Order_ID'].count()
df
```

Out[31]:
```
Traffic_Level
High      197
Low       383
Medium    420
Name: Order_ID, dtype: int64
```

In [32]:
```
data
```

Out[32]:

| | Order_ID | Distance_km | Weather | Traffic_Level | Time_of_Day | Vehicle_Type | Preparation_Time_min | Courier_Experience_yrs |
|---|---|---|---|---|---|---|---|---|
| **0** | 522 | 7.93 | Windy | Low | Afternoon | Scooter | 12 | 1.0 |
| **1** | 738 | 16.42 | Clear | Medium | Evening | Bike | 20 | 2.0 |
| **2** | 741 | 9.52 | Foggy | Low | Night | Scooter | 28 | 1.0 |
| **3** | 661 | 7.44 | Rainy | Medium | Afternoon | Scooter | 5 | 1.0 |
| **4** | 412 | 19.03 | Clear | Low | Morning | Bike | 16 | 5.0 |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... |
| **995** | 107 | 8.50 | Clear | High | Evening | Car | 13 | 3.0 |
| **996** | 271 | 16.28 | Rainy | Low | Morning | Scooter | 8 | 9.0 |
| **997** | 861 | 15.62 | Snowy | High | Evening | Scooter | 26 | 2.0 |
| **998** | 436 | 14.17 | Clear | Low | Afternoon | Bike | 8 | 0.0 |
| **999** | 103 | 6.63 | Foggy | Low | Night | Scooter | 24 | 3.0 |

1000 rows × 9 columns

In [33]:
```
data.columns
```

Out[33]:
```
Index(['Order_ID', 'Distance_km', 'Weather', 'Traffic_Level', 'Time_of_Day',
       'Vehicle_Type', 'Preparation_Time_min', 'Courier_Experience_yrs',
       'Delivery_Time_min'],
      dtype='object')
```

In [34]:
```
df1 = data.groupby('Time_of_Day')['Traffic_Level'].count()
df1
```

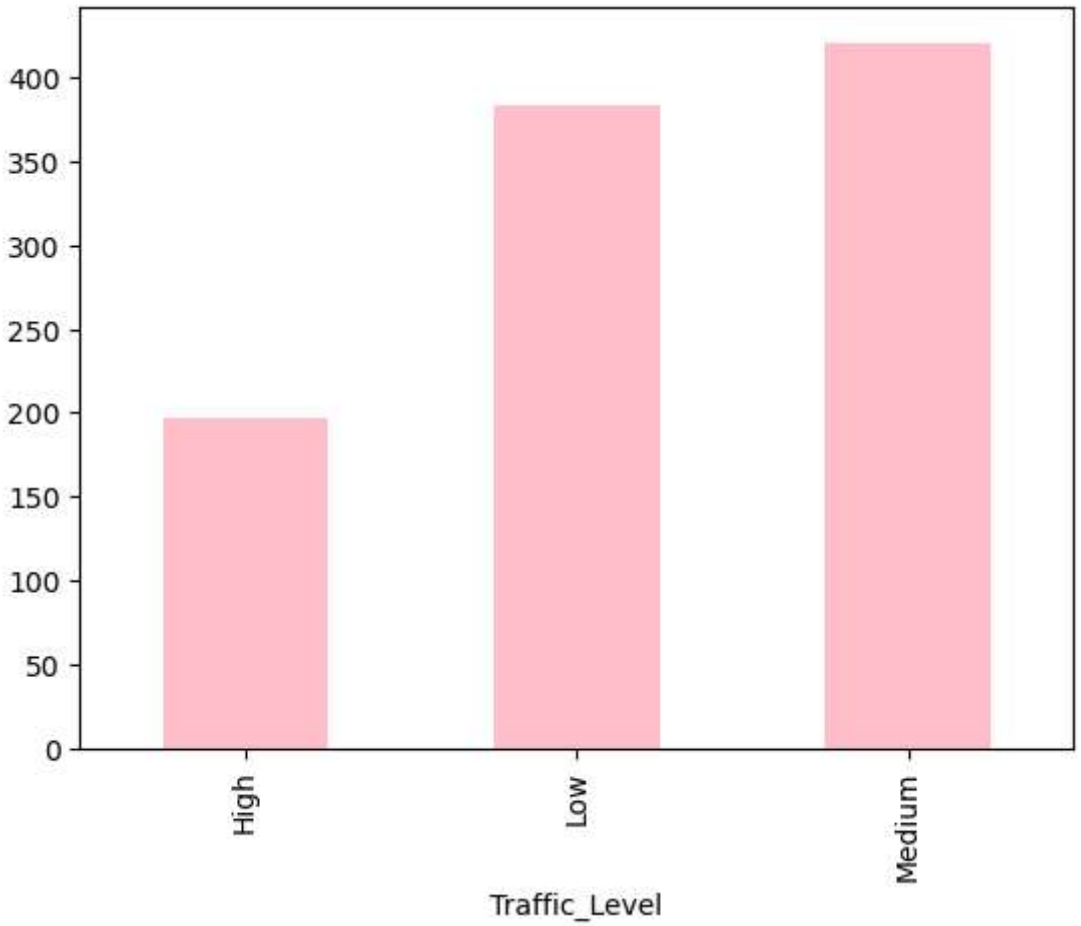
Out[34]:
```
Time_of_Day
Afternoon    284
Evening      293
Morning      338
Night         85
Name: Traffic_Level, dtype: int64
```

# EDA -VISUAL REPRESENTATION

EDA USING PLOTLY

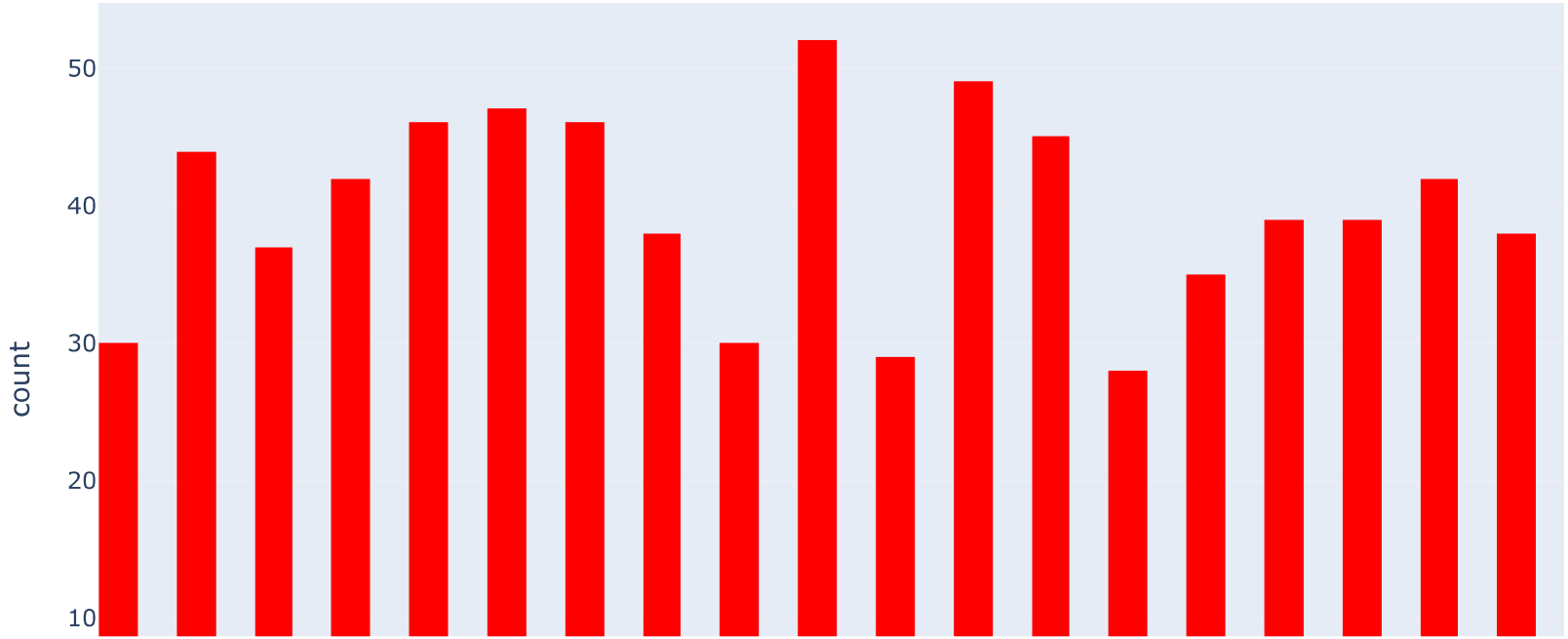
In [35]:
```
fig1 = df.plot(kind = 'bar', color = 'pink')
fig1
```

Out[35]:
```
<Axes: xlabel='Traffic_Level'>
```

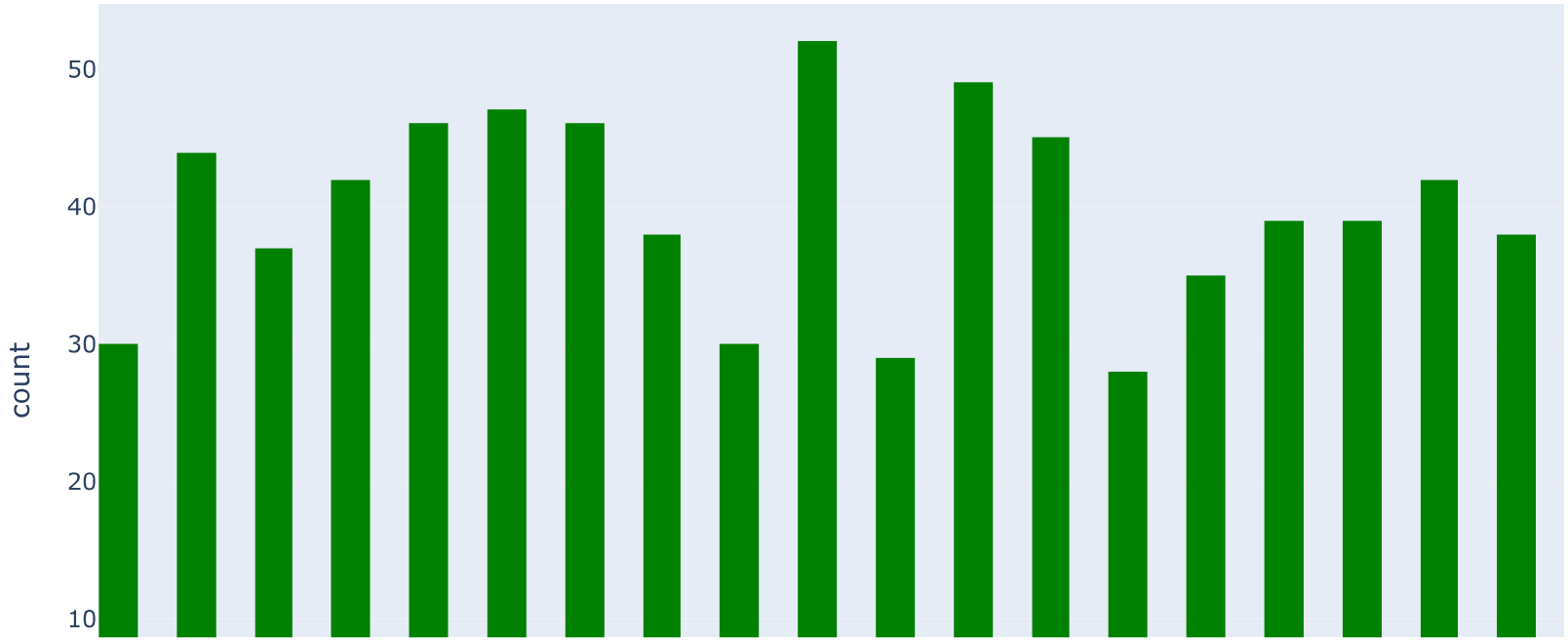


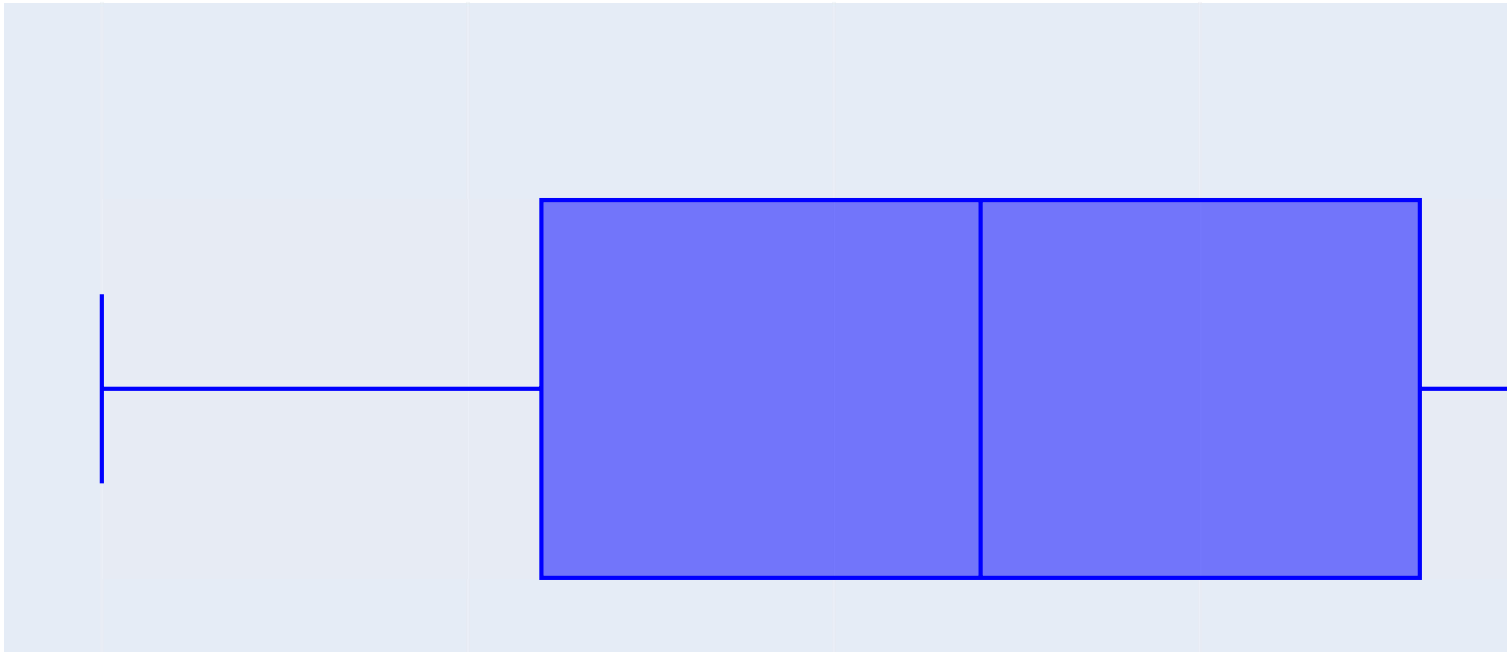

In [36]:
```
fig2 = px.histogram(data, x = 'Distance_km', nbins = 25,color_discrete_sequence=['indianred'])
fig2.show()
```


In [37]:
```
fig3 = px.histogram(data, x = 'Preparation_Time_min', nbins = 50, color_discrete_sequence = ['red'])
fig3.show()
```

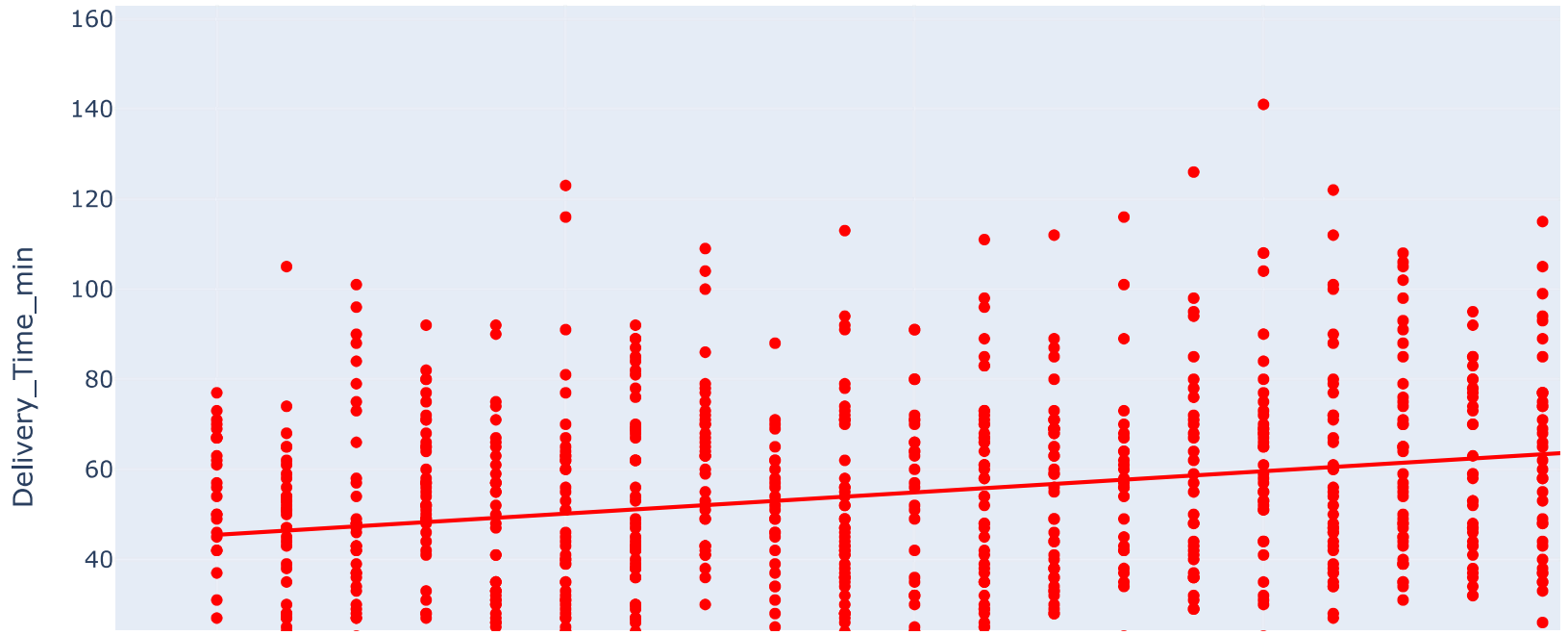



In [38]:
```
fig4 = px.histogram(data, x = 'Preparation_Time_min', nbins = 50, color_discrete_sequence = ['green'])
fig5 = px.box(data, x = 'Preparation_Time_min', color_discrete_sequence = ['blue'])

fig4.show()
fig5.show()
```

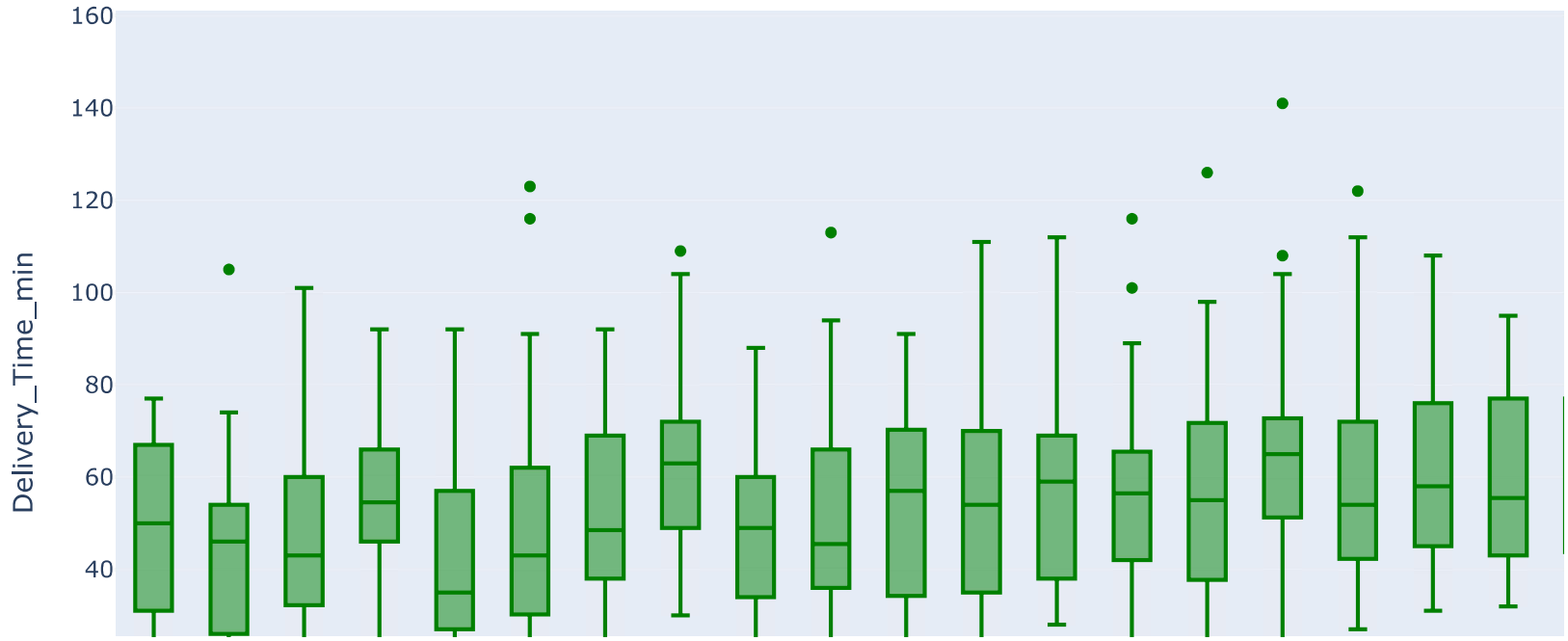

count
50
40
30
20
10



In [39]:
```
fig6 = px.scatter(data, x = 'Preparation_Time_min', y = 'Delivery_Time_min',trendline = 'ols', color_discrete_sequenc
fig6.show()
```




In [40]:
```
fig7 = px.box(data, x = 'Preparation_Time_min', y = 'Delivery_Time_min', color_discrete_sequence = ['green'])
fig7.show()
```

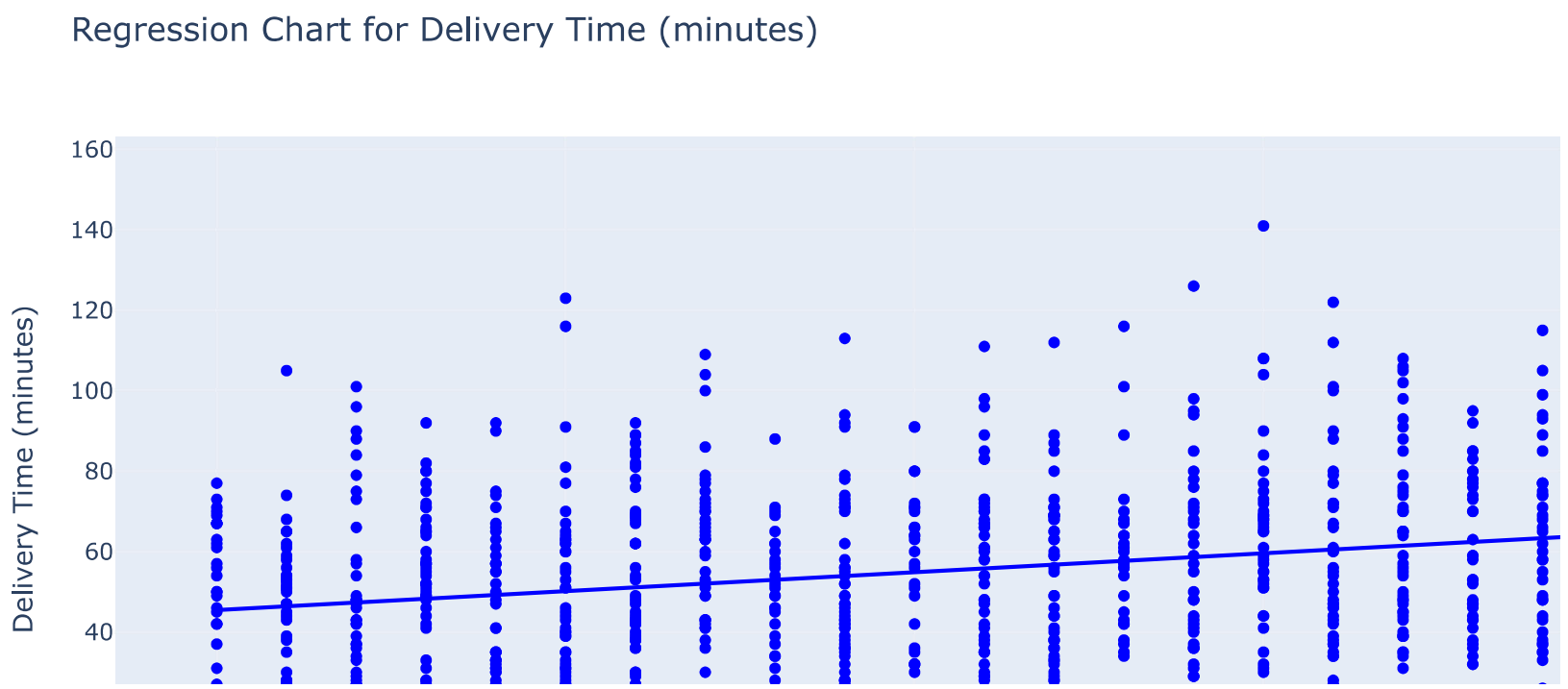



In [41]:
```
data.columns
```

Out[41]:
```
Index(['Order_ID', 'Distance_km', 'Weather', 'Traffic_Level', 'Time_of_Day',
       'Vehicle_Type', 'Preparation_Time_min', 'Courier_Experience_yrs',
       'Delivery_Time_min'],
      dtype='object')
```

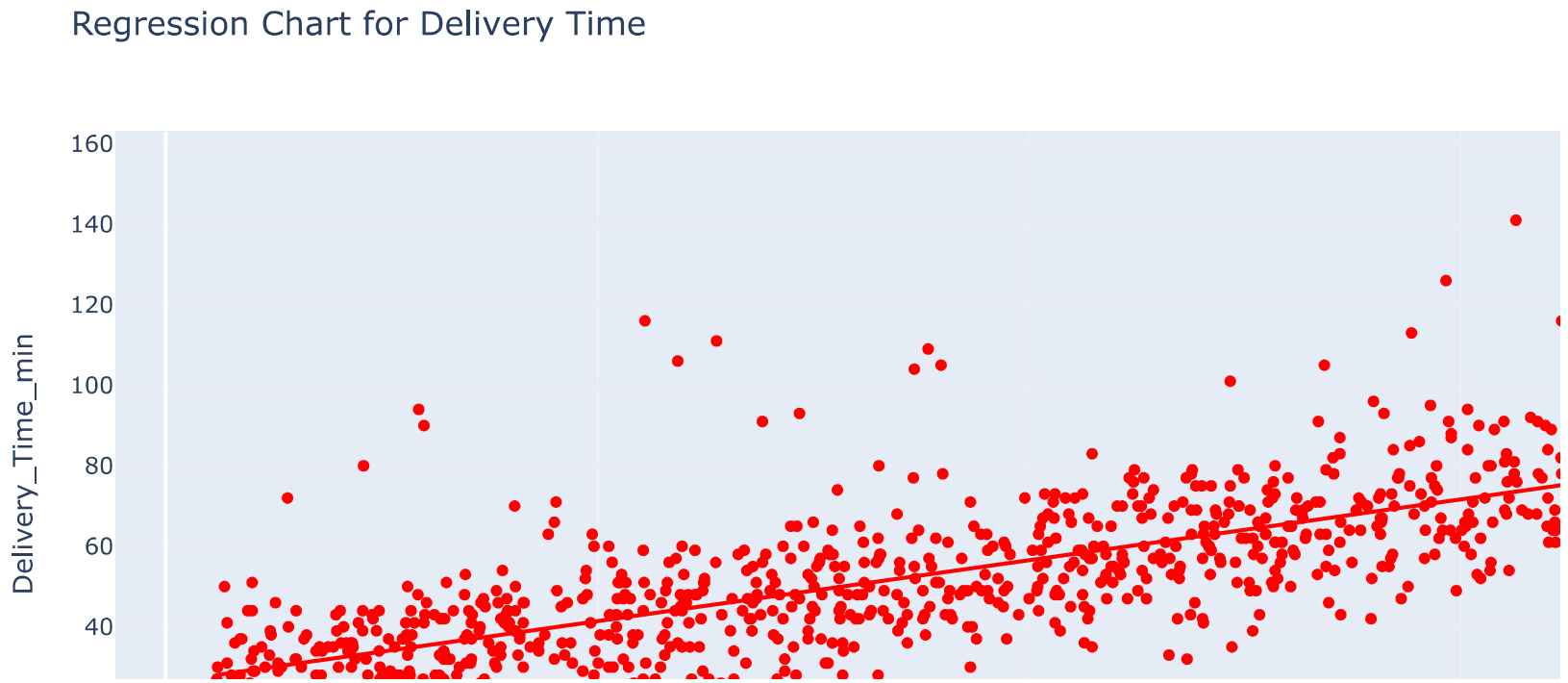
In [42]:
```
fig8= px.scatter(data,
                 x='Preparation_Time_min',
                 y = 'Delivery_Time_min',
```

```
                title = 'Regression Chart for Delivery Time (minutes)',
                labels = {'Preparation_Time_min':'Preparation Time (minutes)',
                          'Delivery_Time_min': 'Delivery Time (minutes)' },
                trendline = 'ols', color_discrete_sequence= ['blue'])

fig8.show()
```




In [43]:
```
fig9 = px.scatter(data,
                  x = 'Distance_km',
                  y = 'Delivery_Time_min',
```

```
                title = 'Regression Chart for Delivery Time',
                trendline = 'ols', color_discrete_sequence = ['red'])

fig9.show()
```



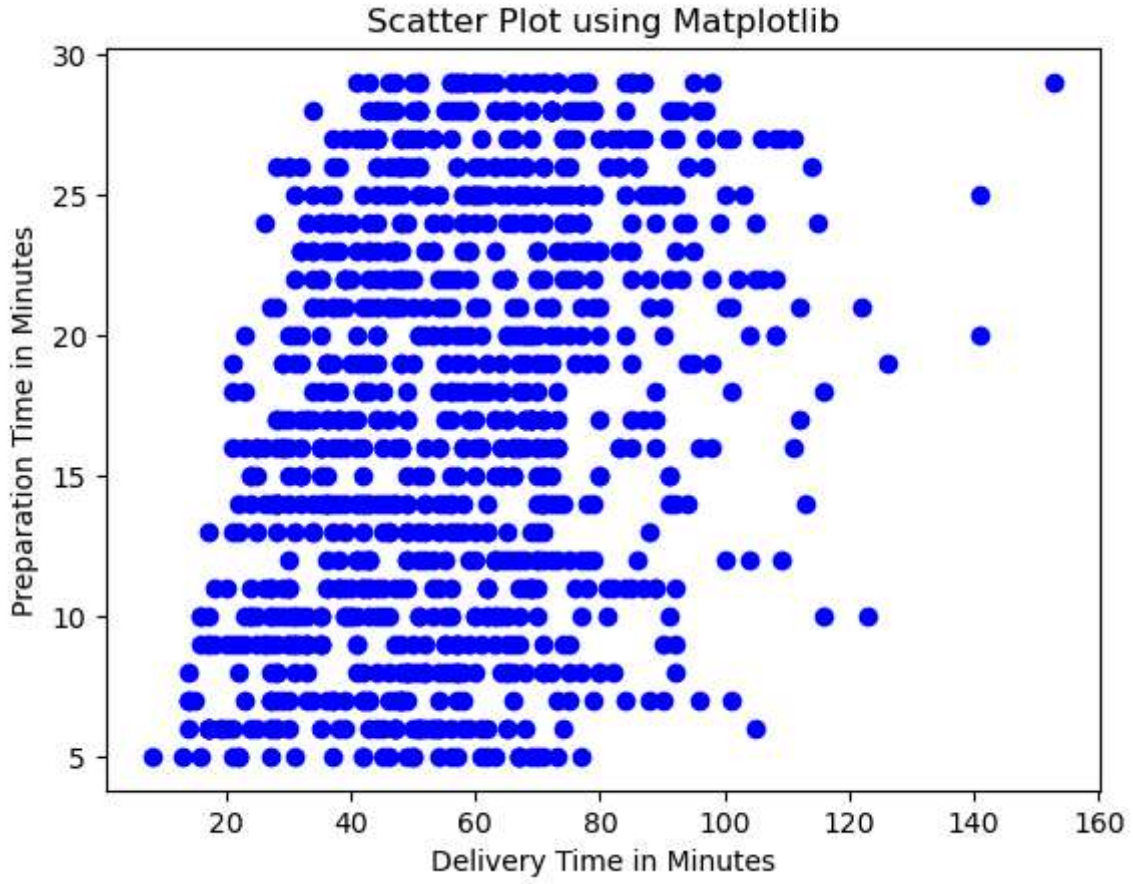

In [44]:
```
data.columns
```

Out[44]:
```
Index(['Order_ID', 'Distance_km', 'Weather', 'Traffic_Level', 'Time_of_Day',
       'Vehicle_Type', 'Preparation_Time_min', 'Courier_Experience_yrs',
       'Delivery_Time_min'],
      dtype='object')
```

EDA USING MATPLOTLIB

In [45]:
```
plt.scatter(data['Delivery_Time_min'], data['Preparation_Time_min'], color = "blue", marker = "o")

plt.title("Scatter Plot using Matplotlib")
plt.xlabel('Delivery Time in Minutes')
plt.ylabel('Preparation Time in Minutes')
```

Out[45]:
```
Text(0, 0.5, 'Preparation Time in Minutes')
```

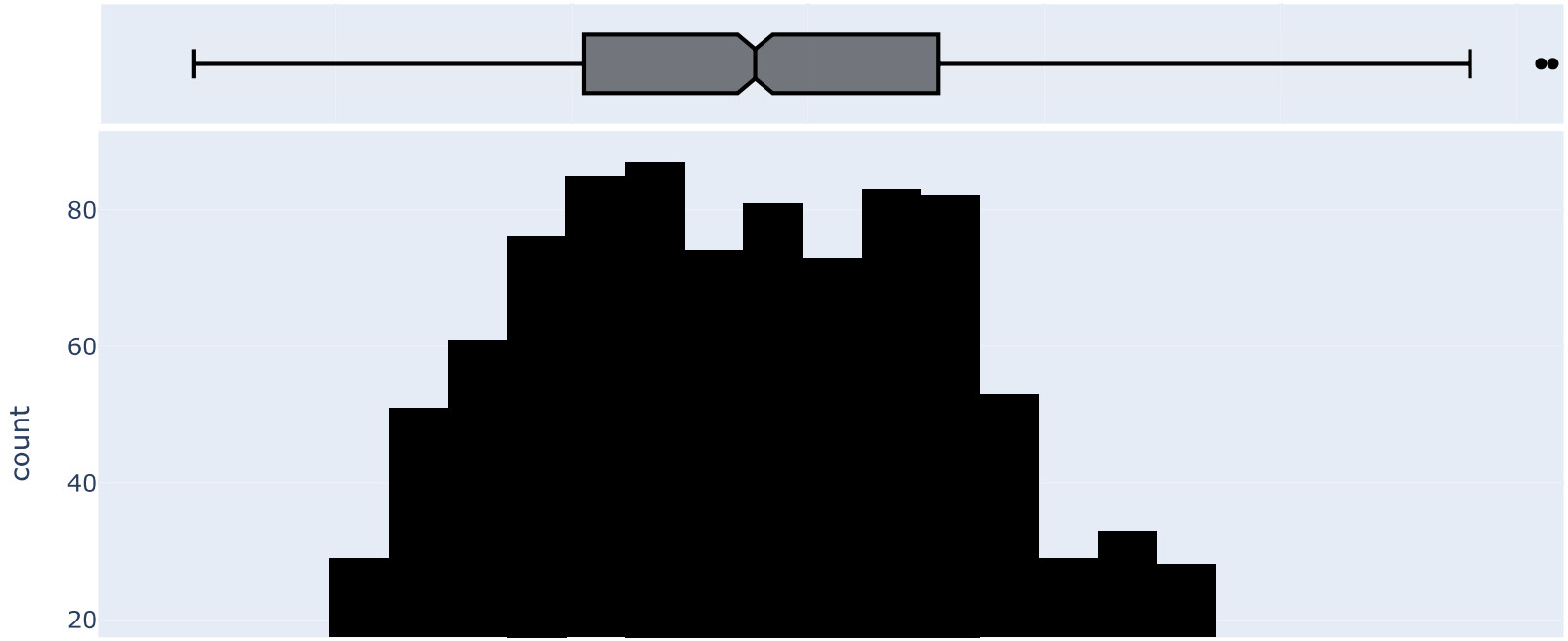


# IDENTIFYING OUTLIERS USING HISTOGRAM

In [46]:
```
data.columns
```

Out[46]:
```
Index(['Order_ID', 'Distance_km', 'Weather', 'Traffic_Level', 'Time_of_Day',
       'Vehicle_Type', 'Preparation_Time_min', 'Courier_Experience_yrs',
       'Delivery_Time_min'],
      dtype='object')
```

In [47]:
```
fig10 = px.histogram(data, x = 'Delivery_Time_min', color_discrete_sequence= ['black'], marginal = 'box')
fig10.show()
```




# Drop Outliers

In [48]:
```
data[data['Delivery_Time_min']>116]

#The above mentioned graph represents an idea of the datapoints of the ouliers i.e.; greater than 116
```

Out[48]:

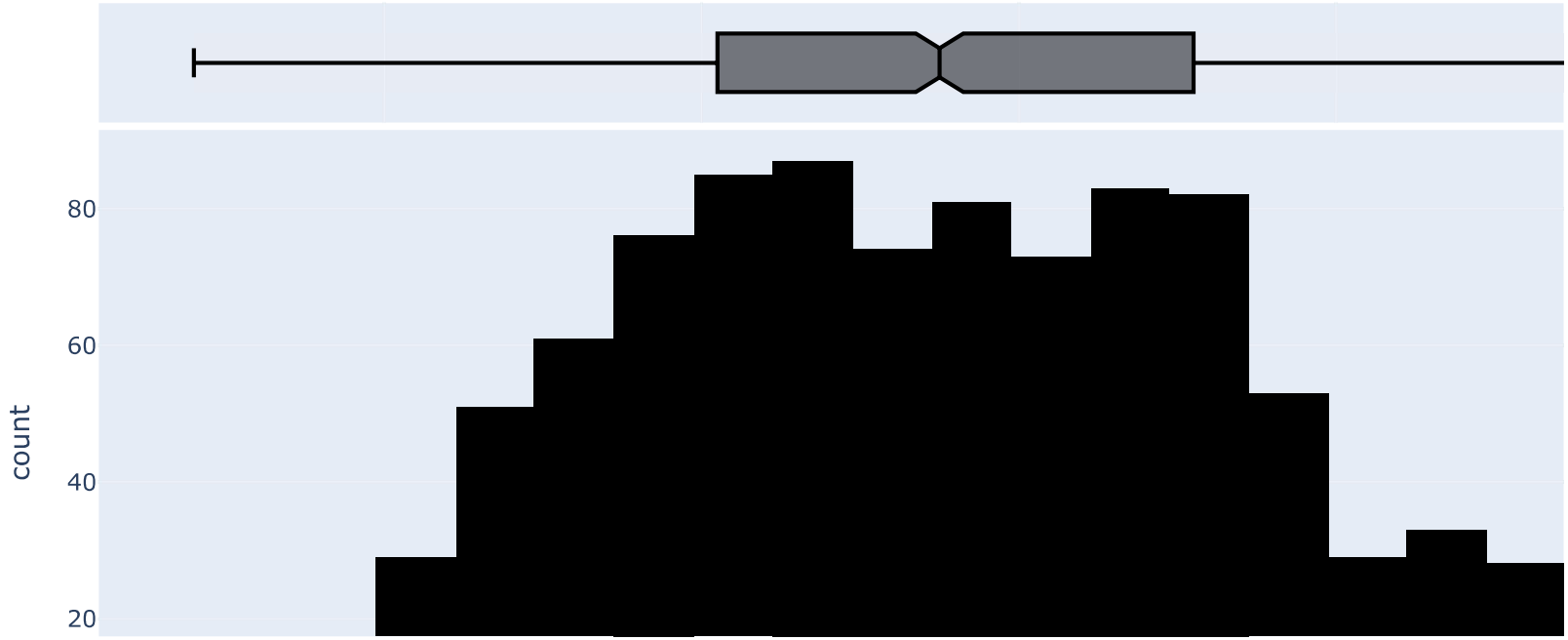
| | Order_ID | Distance_km | Weather | Traffic_Level | Time_of_Day | Vehicle_Type | Preparation_Time_min | Courier_Experience_yrs |
|---|---|---|---|---|---|---|---|---|
| **29** | 948 | 18.05 | Clear | Medium | Evening | Scooter | 10 | 7.0 |
| **127** | 446 | 18.97 | Clear | Low | Evening | Car | 25 | 4.0 |
| **379** | 814 | 18.46 | Clear | Medium | Morning | Scooter | 29 | 1.0 |
| **452** | 394 | 15.64 | Rainy | Low | Morning | Bike | 20 | 4.0 |
| **784** | 385 | 14.83 | Rainy | Low | Morning | Car | 19 | 4.0 |
| **924** | 428 | 17.81 | Windy | High | Evening | Bike | 21 | 4.0 |

In [49]:
```
data[data['Delivery_Time_min']>116].index

#finding out the index of the outliers
```

Out[49]:
```
Index([29, 127, 379, 452, 784, 924], dtype='int64')
```

In [50]:
```
data.drop(data[data['Delivery_Time_min']>116].index, inplace = True)
#dropping outliers
```

In [51]:
```
fig11 = px.histogram(data, x = 'Delivery_Time_min', color_discrete_sequence= ['black'], marginal = 'box')
fig11.show()

#Data after droping outliers
```




SAVE THIS DATASET SEPARATELY IN DIFFERENT CSV FILE

In [52]:
```
data.to_csv('Cleaned_data_food_delivery')
```

# TRAIN- TEST VALIDATION

FOR TRAIN TEST SPLIT/VALIDATION WE WILL IMPORT TRAIN TEST SPLIT FROM SCIKIT-LEARN LIBRARY

In [53]:
```
from sklearn.model_selection import train_test_split
```

In [54]:
```
data.columns
```

Out[54]:
```
Index(['Order_ID', 'Distance_km', 'Weather', 'Traffic_Level', 'Time_of_Day',
       'Vehicle_Type', 'Preparation_Time_min', 'Courier_Experience_yrs',
       'Delivery_Time_min'],
      dtype='object')
```

WE WILL BE DIVIDING THE WHOLE DATA INTO X (feature) columns AND Y (target) column.

In [55]:
```
#define features (X) and target column (Y)

X = data[['Order_ID', 'Distance_km', 'Weather', 'Traffic_Level', 'Time_of_Day',
        'Vehicle_Type', 'Preparation_Time_min', 'Courier_Experience_yrs']]   #dataframe
Y = data['Delivery_Time_min']     #series

#split the dataset
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size = 0.2, random_state = 42)

#printing the split 4 datasets
print(f"X train Shape: {X_train.shape}")
print(f"X test Shape: {X_test.shape}")
print(f"Y train Shape: {Y_train.shape}")
print(f"Y test Shape: {Y_test.shape}")
```

```
X train Shape: (795, 8)
X test Shape: (199, 8)
Y train Shape: (795,)
Y test Shape: (199,)
```

In [56]:
```
X_train
```

Out[56]:

| | Order_ID | Distance_km | Weather | Traffic_Level | Time_of_Day | Vehicle_Type | Preparation_Time_min | Courier_Experience_yrs |
|---|---|---|---|---|---|---|---|---|
| **913** | 566 | 12.00 | Clear | Low | Evening | Car | 25 | 0.0 |
| **930** | 565 | 7.34 | Foggy | Medium | Afternoon | Bike | 22 | 5.0 |
| **664** | 112 | 3.64 | Clear | Low | Morning | Scooter | 16 | 3.0 |
| **768** | 843 | 3.97 | Clear | High | Afternoon | Bike | 19 | 8.0 |
| **915** | 2 | 19.04 | Rainy | Low | Evening | Car | 12 | 6.0 |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... |
| **107** | 960 | 14.89 | Snowy | High | Morning | Scooter | 17 | 6.0 |
| **272** | 901 | 4.55 | Rainy | Low | Afternoon | Bike | 5 | 6.0 |
| **865** | 593 | 4.05 | Clear | Medium | Afternoon | Scooter | 21 | 5.0 |
| **438** | 882 | 13.60 | Foggy | High | Evening | Bike | 9 | 7.0 |
| **103** | 257 | 18.76 | Clear | Medium | Evening | Car | 15 | 2.0 |

795 rows × 8 columns

# FEATURE ENGINEERING

SELECTING RELEVANT COLUMNS FOR MODEL

In [57]:
```
#CREATING A FUNCTION TO DROP UNNECESSARY COLUMNS

def drop_columns(data):
    columns = ['Order_ID', 'Courier_Experience_yrs']
    new_data = data.drop(columns, axis = 1,inplace = True)
    return new_data

drop_columns(X_train)
```

In [58]:
```
X_train
```

Out[58]:

| | Distance_km | Weather | Traffic_Level | Time_of_Day | Vehicle_Type | Preparation_Time_min |
|---|---|---|---|---|---|---|
| **913** | 12.00 | Clear | Low | Evening | Car | 25 |
| **930** | 7.34 | Foggy | Medium | Afternoon | Bike | 22 |
| **664** | 3.64 | Clear | Low | Morning | Scooter | 16 |
| **768** | 3.97 | Clear | High | Afternoon | Bike | 19 |
| **915** | 19.04 | Rainy | Low | Evening | Car | 12 |
| **...** | ... | ... | ... | ... | ... | ... |
| **107** | 14.89 | Snowy | High | Morning | Scooter | 17 |
| **272** | 4.55 | Rainy | Low | Afternoon | Bike | 5 |
| **865** | 4.05 | Clear | Medium | Afternoon | Scooter | 21 |
| **438** | 13.60 | Foggy | High | Evening | Bike | 9 |
| **103** | 18.76 | Clear | Medium | Evening | Car | 15 |

795 rows × 6 columns

# FEATURE EXTRACTION

CREATING NEW FEATURES FROM THE EXISTING FEATURES(COLUMNS)

In [59]:
```
#CREATING A FUNCTION TO CALCULATE SPEED(KM/M)

def create_speed(data):
    data['Speed[Km/m]']= data['Distance_km'] / data['Preparation_Time_min']
    return data
```

In [60]:
```
#Execute the create_speed func() in the X_train dataset

create_speed(X_train)
```

Out[60]:

| | Distance_km | Weather | Traffic_Level | Time_of_Day | Vehicle_Type | Preparation_Time_min | Speed[Km/m] |
|---|---|---|---|---|---|---|---|
| **913** | 12.00 | Clear | Low | Evening | Car | 25 | 0.480000 |
| **930** | 7.34 | Foggy | Medium | Afternoon | Bike | 22 | 0.333636 |
| **664** | 3.64 | Clear | Low | Morning | Scooter | 16 | 0.227500 |
| **768** | 3.97 | Clear | High | Afternoon | Bike | 19 | 0.208947 |
| **915** | 19.04 | Rainy | Low | Evening | Car | 12 | 1.586667 |
| **...** | ... | ... | ... | ... | ... | ... | ... |
| **107** | 14.89 | Snowy | High | Morning | Scooter | 17 | 0.875882 |
| **272** | 4.55 | Rainy | Low | Afternoon | Bike | 5 | 0.910000 |
| **865** | 4.05 | Clear | Medium | Afternoon | Scooter | 21 | 0.192857 |
| **438** | 13.60 | Foggy | High | Evening | Bike | 9 | 1.511111 |
| **103** | 18.76 | Clear | Medium | Evening | Car | 15 | 1.250667 |

795 rows × 7 columns

In [61]:
```
X_train
```

Out[61]:

| | Distance_km | Weather | Traffic_Level | Time_of_Day | Vehicle_Type | Preparation_Time_min | Speed[Km/m] |
|---|---|---|---|---|---|---|---|
| **913** | 12.00 | Clear | Low | Evening | Car | 25 | 0.480000 |
| **930** | 7.34 | Foggy | Medium | Afternoon | Bike | 22 | 0.333636 |
| **664** | 3.64 | Clear | Low | Morning | Scooter | 16 | 0.227500 |
| **768** | 3.97 | Clear | High | Afternoon | Bike | 19 | 0.208947 |
| **915** | 19.04 | Rainy | Low | Evening | Car | 12 | 1.586667 |
| **...** | ... | ... | ... | ... | ... | ... | ... |
| **107** | 14.89 | Snowy | High | Morning | Scooter | 17 | 0.875882 |
| **272** | 4.55 | Rainy | Low | Afternoon | Bike | 5 | 0.910000 |
| **865** | 4.05 | Clear | Medium | Afternoon | Scooter | 21 | 0.192857 |
| **438** | 13.60 | Foggy | High | Evening | Bike | 9 | 1.511111 |
| **103** | 18.76 | Clear | Medium | Evening | Car | 15 | 1.250667 |

795 rows × 7 columns

# FEATURE ENCODING & FEATURE SCALING

FEATURE ENCODING - USING ONE HOT ENCODER FOR CATEGORICAL COLUMNS

FEATURE SCALING - USING STANDARD SCALER FOR NUMERICAL COLUMNS

USING COLUMN TRANSFORMER FOR BOTH THE METHODS

In [62]:
```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler,OneHotEncoder
```

In [63]:
```
X_train.columns
```

Out[63]:
```
Index(['Distance_km', 'Weather', 'Traffic_Level', 'Time_of_Day',
       'Vehicle_Type', 'Preparation_Time_min', 'Speed[Km/m]'],
      dtype='object')
```

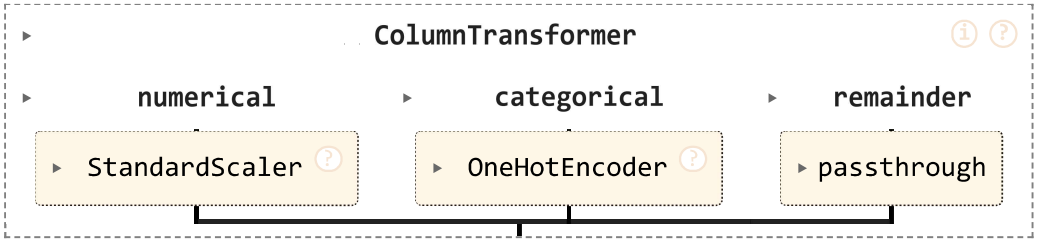
In [64]:
```
X_train.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 795 entries, 913 to 103
Data columns (total 7 columns):
 #   Column                Non-Null Count  Dtype  
---  ------                --------------  -----  
 0   Distance_km           795 non-null    float64
 1   Weather               795 non-null    object 
 2   Traffic_Level         795 non-null    object 
 3   Time_of_Day           795 non-null    object 
 4   Vehicle_Type          795 non-null    object 
 5   Preparation_Time_min  795 non-null    int64  
 6   Speed[Km/m]           795 non-null    float64
dtypes: float64(2), int64(1), object(4)
memory usage: 49.7+ KB
```

In [65]:
```
transformer = ColumnTransformer(
    transformers = [('numerical', StandardScaler(), ['Distance_km','Preparation_Time_min',
                    'Speed[Km/m]']),
                    ('categorical', OneHotEncoder(),['Weather','Traffic_Level',
                     'Time_of_Day','Vehicle_Type'])
                    ],
    remainder = 'passthrough'

)
```

In [66]:
```
transformer
```

Out[66]:



USING FIR & TRANSFORM OVER X_TRAIN DATASET

In [67]:
```
new_X_train = transformer.fit_transform(X_train)
```

In [68]:
```
new_X_train
```

Out[68]:
```
array([[ 0.31523378,  1.10156557, -0.43818825, ...,  0.        ,
         1.        ,  0.        ],
       [-0.51235709,  0.68756716, -0.64864984, ...,  1.        ,
         0.        ,  0.        ],
       [-1.16945714, -0.14042965, -0.80126718, ...,  0.        ,
         0.        ,  1.        ],
       ...,
       [-1.09664335,  0.54956769, -0.8510814 , ...,  0.        ,
         0.        ,  1.        ],
       [ 0.59938515, -1.10642592,  1.04448387, ...,  1.        ,
         0.        ,  0.        ],
       [ 1.51577332, -0.27842912,  0.66998135, ...,  0.        ,
         1.        ,  0.        ]])
```

GETTING THE NEAMES OF THE FEATURES AFTER APPLYING COLUMN TRANSFORMER

In [69]:
```
transformer.named_transformers_['numerical'].get_feature_names_out()
```

Out[69]:
```
array(['Distance_km', 'Preparation_Time_min', 'Speed[Km/m]'], dtype=object)
```

In [70]:
```
transformer.named_transformers_['categorical'].get_feature_names_out()
```

Out[70]:
```
array(['Weather_Clear', 'Weather_Foggy', 'Weather_Rainy', 'Weather_Snowy',
       'Weather_Windy', 'Traffic_Level_High', 'Traffic_Level_Low',
       'Traffic_Level_Medium', 'Time_of_Day_Afternoon',
       'Time_of_Day_Evening', 'Time_of_Day_Morning', 'Time_of_Day_Night',
       'Vehicle_Type_Bike', 'Vehicle_Type_Car', 'Vehicle_Type_Scooter'],
      dtype=object)
```

CONVERTING THIS ARRAY INTO A DATAFRAME WITH THE FEATURES NAMES

In [71]:
```
X_train_Final = pd.DataFrame(new_X_train, columns = ['Distance_km', 'Preparation_Time_min',
                                                     'Speed[Km/m]', 'Weather_Clear', 'Weather_Foggy',
       'Weather_Rainy', 'Weather_Snowy',
```

```
       'Weather_Windy', 'Traffic_Level_High', 'Traffic_Level_Low',
       'Traffic_Level_Medium', 'Time_of_Day_Afternoon',
       'Time_of_Day_Evening', 'Time_of_Day_Morning', 'Time_of_Day_Night',
       'Vehicle_Type_Bike', 'Vehicle_Type_Car', 'Vehicle_Type_Scooter'])
```

In [72]:
```
X_train_Final
```

Out[72]:

| | Distance_km | Preparation_Time_min | Speed[Km/m] | Weather_Clear | Weather_Foggy | Weather_Rainy | Weather_Snowy | Weatl |
|---|---|---|---|---|---|---|---|---|
| **0** | 0.315234 | 1.101566 | -0.438188 | 1.0 | 0.0 | 0.0 | 0.0 | |
| **1** | -0.512357 | 0.687567 | -0.648650 | 0.0 | 1.0 | 0.0 | 0.0 | |
| **2** | -1.169457 | -0.140430 | -0.801267 | 1.0 | 0.0 | 0.0 | 0.0 | |
| **3** | -1.110851 | 0.273569 | -0.827945 | 1.0 | 0.0 | 0.0 | 0.0 | |
| **4** | 1.565500 | -0.692428 | 1.153128 | 0.0 | 0.0 | 1.0 | 0.0 | |
| **...** | ... | ... | ... | ... | ... | ... | ... | |
| **790** | 0.828482 | -0.002430 | 0.131065 | 0.0 | 0.0 | 0.0 | 1.0 | |
| **791** | -1.007846 | -1.658424 | 0.180124 | 0.0 | 0.0 | 1.0 | 0.0 | |
| **792** | -1.096643 | 0.549568 | -0.851081 | 1.0 | 0.0 | 0.0 | 0.0 | |
| **793** | 0.599385 | -1.106426 | 1.044484 | 0.0 | 1.0 | 0.0 | 0.0 | |
| **794** | 1.515773 | -0.278429 | 0.669981 | 1.0 | 0.0 | 0.0 | 0.0 | |

795 rows × 18 columns

In [73]:
```
X_train_Final.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 795 entries, 0 to 794
Data columns (total 18 columns):
 #   Column                 Non-Null Count  Dtype  
---  ------                 --------------  -----  
 0   Distance_km            795 non-null    float64
 1   Preparation_Time_min   795 non-null    float64
 2   Speed[Km/m]            795 non-null    float64
 3   Weather_Clear          795 non-null    float64
 4   Weather_Foggy          795 non-null    float64
 5   Weather_Rainy          795 non-null    float64
 6   Weather_Snowy          795 non-null    float64
 7   Weather_Windy          795 non-null    float64
 8   Traffic_Level_High     795 non-null    float64
 9   Traffic_Level_Low      795 non-null    float64
 10  Traffic_Level_Medium   795 non-null    float64
 11  Time_of_Day_Afternoon  795 non-null    float64
 12  Time_of_Day_Evening    795 non-null    float64
 13  Time_of_Day_Morning    795 non-null    float64
 14  Time_of_Day_Night      795 non-null    float64
 15  Vehicle_Type_Bike      795 non-null    float64
 16  Vehicle_Type_Car       795 non-null    float64
 17  Vehicle_Type_Scooter   795 non-null    float64
dtypes: float64(18)
memory usage: 111.9 KB
```

In [74]:
```
X_train_Final.shape
```

Out[74]:
```
(795, 18)
```

In [75]:
```
X_train_Final.columns
```

Out[75]:
```
Index(['Distance_km', 'Preparation_Time_min', 'Speed[Km/m]', 'Weather_Clear',
       'Weather_Foggy', 'Weather_Rainy', 'Weather_Snowy', 'Weather_Windy',
       'Traffic_Level_High', 'Traffic_Level_Low', 'Traffic_Level_Medium',
       'Time_of_Day_Afternoon', 'Time_of_Day_Evening', 'Time_of_Day_Morning',
       'Time_of_Day_Night', 'Vehicle_Type_Bike', 'Vehicle_Type_Car',
       'Vehicle_Type_Scooter'],
      dtype='object')
```

# FEATURE SELECTION

DROPPING SOME IRRELEVANT COLUMNS AND CHOOSING RELEVANT FEATURES

In [76]:
```
#creating functions for dropping unncessary columns

def unselected_feature(data):
    columns = ['Weather_Foggy','Weather_Snowy']
    new_data = data.drop(columns, axis = 1,inplace = True)
    return new_data

unselected_feature(X_train_Final)
```

In [77]:
```
def unselected_feature_new(data):
    columns = ['Traffic_Level_Medium','Vehicle_Type_Scooter','Time_of_Day_Night']
    new_data = data.drop(columns, axis = 1,inplace = True)
    return new_data

unselected_feature_new(X_train_Final)
```

In [78]:
```
X_train_Final
```

Out[78]:

| | Distance_km | Preparation_Time_min | Speed[Km/m] | Weather_Clear | Weather_Rainy | Weather_Windy | Traffic_Level_High | Tra |
|---|---|---|---|---|---|---|---|---|
| **0** | 0.315234 | 1.101566 | -0.438188 | 1.0 | 0.0 | 0.0 | 0.0 | |
| **1** | -0.512357 | 0.687567 | -0.648650 | 0.0 | 0.0 | 0.0 | 0.0 | |
| **2** | -1.169457 | -0.140430 | -0.801267 | 1.0 | 0.0 | 0.0 | 0.0 | |
| **3** | -1.110851 | 0.273569 | -0.827945 | 1.0 | 0.0 | 0.0 | 1.0 | |
| **4** | 1.565500 | -0.692428 | 1.153128 | 0.0 | 1.0 | 0.0 | 0.0 | |
| **...** | ... | ... | ... | ... | ... | ... | ... | |
| **790** | 0.828482 | -0.002430 | 0.131065 | 0.0 | 0.0 | 0.0 | 1.0 | |
| **791** | -1.007846 | -1.658424 | 0.180124 | 0.0 | 1.0 | 0.0 | 0.0 | |
| **792** | -1.096643 | 0.549568 | -0.851081 | 1.0 | 0.0 | 0.0 | 0.0 | |
| **793** | 0.599385 | -1.106426 | 1.044484 | 0.0 | 0.0 | 0.0 | 1.0 | |
| **794** | 1.515773 | -0.278429 | 0.669981 | 1.0 | 0.0 | 0.0 | 0.0 | |

795 rows × 13 columns

In [79]:
```
Y_train
```

Out[79]:
```
913    75
930    93
664    26
768    40
915    60
       ..
107    87
272    21
865    39
438    61
103    80
Name: Delivery_Time_min, Length: 795, dtype: int64
```

In [80]:
```
#saving the Y_train dataset as a new dataframe


Y_train.to_csv('Y_train_Final')
```

In [81]:
```
#retrieving the Y_train dataset as Y_train_Final dataframe

Y_train_Final = pd.read_csv("Y_train_Final", index_col = 0)
```

In [82]:
```
Y_train_Final
```

Out[82]:

| | Delivery_Time_min |
|---|---|
| **913** | 75 |
| **930** | 93 |
| **664** | 26 |
| **768** | 40 |
| **915** | 60 |
| **...** | ... |
| **107** | 87 |
| **272** | 21 |
| **865** | 39 |
| **438** | 61 |
| **103** | 80 |

795 rows × 1 columns

# MODEL BUILDING

IMPORTING LINEAR REGRESSION MODEL FROM SCI-KIT LEARN LINEAR MODEL LIBRARIES

In [83]:
```
from sklearn.linear_model import LinearRegression
```

In [84]:
```
#model selection

model = LinearRegression()
```

In [85]:
```
#fitting Linear Regression model for the dataset

model.fit(X_train_Final, Y_train_Final)
```

Out[85]:
```
LinearRegression
LinearRegression()
```

In [86]:
```
#testing model accuracy on Training dataset

model.score(X_train_Final, Y_train_Final)
```

Out[86]: 0.774484924647839

# TESTING DATASET

In [87]:
```
X_test
```

Out[87]:

| | Order_ID | Distance_km | Weather | Traffic_Level | Time_of_Day | Vehicle_Type | Preparation_Time_min | Courier_Experience_yrs |
|---|---|---|---|---|---|---|---|---|
| **925** | 455 | 2.67 | Clear | Medium | Night | Car | 17 | 6.0 |
| **529** | 94 | 11.44 | Rainy | Medium | Evening | Car | 20 | 2.0 |
| **571** | 125 | 4.94 | Rainy | High | Evening | Scooter | 28 | 4.0 |
| **660** | 153 | 3.33 | Clear | Medium | Evening | Scooter | 24 | 2.0 |
| **932** | 777 | 4.97 | Rainy | Low | Evening | Bike | 17 | 9.0 |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... |
| **490** | 877 | 14.46 | Rainy | Medium | Morning | Scooter | 11 | 9.0 |
| **455** | 635 | 2.59 | Windy | Medium | Morning | Bike | 13 | 5.0 |
| **66** | 663 | 12.31 | Windy | Low | Morning | Bike | 29 | 4.0 |
| **143** | 282 | 7.09 | Snowy | Low | Morning | Scooter | 13 | 8.0 |
| **688** | 254 | 6.68 | Clear | High | Morning | Scooter | 10 | 5.0 |

199 rows × 8 columns

In [88]:
```
#saving the X_test dataset into new dataframe

X_test.to_csv('X_test_new')
```

In [89]:
```
#retrieving the X_test dataset

X_test_new = pd.read_csv('X_test_new', index_col =0)
```

In [90]:
```
X_test_new
```

Out[90]:

| | Order_ID | Distance_km | Weather | Traffic_Level | Time_of_Day | Vehicle_Type | Preparation_Time_min | Courier_Experience_yrs |
|---|---|---|---|---|---|---|---|---|
| **925** | 455 | 2.67 | Clear | Medium | Night | Car | 17 | 6.0 |
| **529** | 94 | 11.44 | Rainy | Medium | Evening | Car | 20 | 2.0 |
| **571** | 125 | 4.94 | Rainy | High | Evening | Scooter | 28 | 4.0 |
| **660** | 153 | 3.33 | Clear | Medium | Evening | Scooter | 24 | 2.0 |
| **932** | 777 | 4.97 | Rainy | Low | Evening | Bike | 17 | 9.0 |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... |
| **490** | 877 | 14.46 | Rainy | Medium | Morning | Scooter | 11 | 9.0 |
| **455** | 635 | 2.59 | Windy | Medium | Morning | Bike | 13 | 5.0 |
| **66** | 663 | 12.31 | Windy | Low | Morning | Bike | 29 | 4.0 |
| **143** | 282 | 7.09 | Snowy | Low | Morning | Scooter | 13 | 8.0 |
| **688** | 254 | 6.68 | Clear | High | Morning | Scooter | 10 | 5.0 |

199 rows × 8 columns

DROPPINH COLUMNS USING FUNCTIONS

In [91]:
```
drop_columns(X_test_new)
```

In [92]:
```
X_test_new
```

Out[92]:

| | Distance_km | Weather | Traffic_Level | Time_of_Day | Vehicle_Type | Preparation_Time_min |
|---|---|---|---|---|---|---|
| **925** | 2.67 | Clear | Medium | Night | Car | 17 |
| **529** | 11.44 | Rainy | Medium | Evening | Car | 20 |
| **571** | 4.94 | Rainy | High | Evening | Scooter | 28 |
| **660** | 3.33 | Clear | Medium | Evening | Scooter | 24 |
| **932** | 4.97 | Rainy | Low | Evening | Bike | 17 |
| **...** | ... | ... | ... | ... | ... | ... |
| **490** | 14.46 | Rainy | Medium | Morning | Scooter | 11 |
| **455** | 2.59 | Windy | Medium | Morning | Bike | 13 |
| **66** | 12.31 | Windy | Low | Morning | Bike | 29 |
| **143** | 7.09 | Snowy | Low | Morning | Scooter | 13 |
| **688** | 6.68 | Clear | High | Morning | Scooter | 10 |

199 rows × 6 columns

CALCULATING SPEED

In [93]:
```
create_speed(X_test_new)
```

Out[93]:

| | Distance_km | Weather | Traffic_Level | Time_of_Day | Vehicle_Type | Preparation_Time_min | Speed[Km/m] |
|---|---|---|---|---|---|---|---|
| **925** | 2.67 | Clear | Medium | Night | Car | 17 | 0.157059 |
| **529** | 11.44 | Rainy | Medium | Evening | Car | 20 | 0.572000 |
| **571** | 4.94 | Rainy | High | Evening | Scooter | 28 | 0.176429 |
| **660** | 3.33 | Clear | Medium | Evening | Scooter | 24 | 0.138750 |
| **932** | 4.97 | Rainy | Low | Evening | Bike | 17 | 0.292353 |
| **...** | ... | ... | ... | ... | ... | ... | ... |
| **490** | 14.46 | Rainy | Medium | Morning | Scooter | 11 | 1.314545 |
| **455** | 2.59 | Windy | Medium | Morning | Bike | 13 | 0.199231 |
| **66** | 12.31 | Windy | Low | Morning | Bike | 29 | 0.424483 |
| **143** | 7.09 | Snowy | Low | Morning | Scooter | 13 | 0.545385 |
| **688** | 6.68 | Clear | High | Morning | Scooter | 10 | 0.668000 |

199 rows × 7 columns

APPLYING COLUMN TRANSFORMER

In [94]:
```
new_X_test = transformer.transform(X_test_new)
```

In [95]:
```
new_X_test
```

Out[95]:
```
array([[-1.3417239 , -0.00243018, -0.9025571 , ...,  0.        ,
         1.        ,  0.        ],
       [ 0.2157808 ,  0.41156822, -0.30589811, ...,  0.        ,
         1.        ,  0.        ],
       [-0.93858415,  1.51556397, -0.87470464, ...,  0.        ,
         0.        ,  1.        ],
       ...,
       [ 0.37028811,  1.65356344, -0.51801851, ...,  1.        ,
         0.        ,  0.        ],
       [-0.55675574, -0.55442805, -0.34416934, ...,  0.        ,
         0.        ,  1.        ],
       [-0.62956953, -0.96842646, -0.16785622, ...,  0.        ,
         0.        ,  1.        ]])
```

GEETING FEATURES NAMES OUT

In [96]:
```
transformer.named_transformers_['categorical'].get_feature_names_out()
```

Out[96]:
```
array(['Weather_Clear', 'Weather_Foggy', 'Weather_Rainy', 'Weather_Snowy',
       'Weather_Windy', 'Traffic_Level_High', 'Traffic_Level_Low',
       'Traffic_Level_Medium', 'Time_of_Day_Afternoon',
       'Time_of_Day_Evening', 'Time_of_Day_Morning', 'Time_of_Day_Night',
       'Vehicle_Type_Bike', 'Vehicle_Type_Car', 'Vehicle_Type_Scooter'],
      dtype=object)
```

In [97]:
```
transformer.named_transformers_['numerical'].get_feature_names_out()
```

Out[97]:
```
array(['Distance_km', 'Preparation_Time_min', 'Speed[Km/m]'], dtype=object)
```

In [98]:
```
X_test_Final = pd.DataFrame(new_X_test, columns = ['Distance_km', 'Preparation_Time_min', 'Speed[Km/m]',
                                                   'Weather_Clear', 'Weather_Foggy', 'Weather_Rainy', 'Weather_Snowy',
       'Weather_Windy', 'Traffic_Level_High', 'Traffic_Level_Low',
       'Traffic_Level_Medium', 'Time_of_Day_Afternoon',
       'Time_of_Day_Evening', 'Time_of_Day_Morning', 'Time_of_Day_Night',
       'Vehicle_Type_Bike', 'Vehicle_Type_Car', 'Vehicle_Type_Scooter' ])
```

In [99]:
```
X_test_Final
```

Out[99]:

| | Distance_km | Preparation_Time_min | Speed[Km/m] | Weather_Clear | Weather_Foggy | Weather_Rainy | Weather_Snowy | Weatl |
|---|---|---|---|---|---|---|---|---|
| **0** | -1.341724 | -0.002430 | -0.902557 | 1.0 | 0.0 | 0.0 | 0.0 | |
| **1** | 0.215781 | 0.411568 | -0.305898 | 0.0 | 0.0 | 1.0 | 0.0 | |
| **2** | -0.938584 | 1.515564 | -0.874705 | 0.0 | 0.0 | 1.0 | 0.0 | |
| **3** | -1.224511 | 0.963566 | -0.928884 | 1.0 | 0.0 | 0.0 | 0.0 | |
| **4** | -0.933256 | -0.002430 | -0.708013 | 0.0 | 0.0 | 1.0 | 0.0 | |
| **...** | ... | ... | ... | ... | ... | ... | ... | |
| **194** | 0.752117 | -0.830427 | 0.761835 | 0.0 | 0.0 | 1.0 | 0.0 | |
| **195** | -1.355931 | -0.554428 | -0.841917 | 0.0 | 0.0 | 0.0 | 0.0 | |
| **196** | 0.370288 | 1.653563 | -0.518019 | 0.0 | 0.0 | 0.0 | 0.0 | |
| **197** | -0.556756 | -0.554428 | -0.344169 | 0.0 | 0.0 | 0.0 | 1.0 | |
| **198** | -0.629570 | -0.968426 | -0.167856 | 1.0 | 0.0 | 0.0 | 0.0 | |

199 rows × 18 columns

FEATURE SELECTION

In [100…
```
#dropping columns using user-defined functions

unselected_feature(X_test_Final)
```

In [101…
```
#dropping columns

unselected_feature_new(X_test_Final)
```

In [102…
```
#Final X_test dataset

X_test_Final
```

Out[102…

| | Distance_km | Preparation_Time_min | Speed[Km/m] | Weather_Clear | Weather_Rainy | Weather_Windy | Traffic_Level_High | Tra |
|---|---|---|---|---|---|---|---|---|
| **0** | -1.341724 | -0.002430 | -0.902557 | 1.0 | 0.0 | 0.0 | 0.0 | |
| **1** | 0.215781 | 0.411568 | -0.305898 | 0.0 | 1.0 | 0.0 | 0.0 | |
| **2** | -0.938584 | 1.515564 | -0.874705 | 0.0 | 1.0 | 0.0 | 1.0 | |
| **3** | -1.224511 | 0.963566 | -0.928884 | 1.0 | 0.0 | 0.0 | 0.0 | |
| **4** | -0.933256 | -0.002430 | -0.708013 | 0.0 | 1.0 | 0.0 | 0.0 | |
| **…** | … | … | … | … | … | … | … | |
| **194** | 0.752117 | -0.830427 | 0.761835 | 0.0 | 1.0 | 0.0 | 0.0 | |
| **195** | -1.355931 | -0.554428 | -0.841917 | 0.0 | 0.0 | 1.0 | 0.0 | |
| **196** | 0.370288 | 1.653563 | -0.518019 | 0.0 | 0.0 | 1.0 | 0.0 | |
| **197** | -0.556756 | -0.554428 | -0.344169 | 0.0 | 0.0 | 0.0 | 0.0 | |
| **198** | -0.629570 | -0.968426 | -0.167856 | 1.0 | 0.0 | 0.0 | 1.0 | |

199 rows × 13 columns

In [103…

```
X_train_Final
```

Out[103…

| | Distance_km | Preparation_Time_min | Speed[Km/m] | Weather_Clear | Weather_Rainy | Weather_Windy | Traffic_Level_High | Tra |
|---|---|---|---|---|---|---|---|---|
| **0** | 0.315234 | 1.101566 | -0.438188 | 1.0 | 0.0 | 0.0 | 0.0 | |
| **1** | -0.512357 | 0.687567 | -0.648650 | 0.0 | 0.0 | 0.0 | 0.0 | |
| **2** | -1.169457 | -0.140430 | -0.801267 | 1.0 | 0.0 | 0.0 | 0.0 | |
| **3** | -1.110851 | 0.273569 | -0.827945 | 1.0 | 0.0 | 0.0 | 1.0 | |
| **4** | 1.565500 | -0.692428 | 1.153128 | 0.0 | 1.0 | 0.0 | 0.0 | |
| **…** | … | … | … | … | … | … | … | |
| **790** | 0.828482 | -0.002430 | 0.131065 | 0.0 | 0.0 | 0.0 | 1.0 | |
| **791** | -1.007846 | -1.658424 | 0.180124 | 0.0 | 1.0 | 0.0 | 0.0 | |
| **792** | -1.096643 | 0.549568 | -0.851081 | 1.0 | 0.0 | 0.0 | 0.0 | |
| **793** | 0.599385 | -1.106426 | 1.044484 | 0.0 | 0.0 | 0.0 | 1.0 | |
| **794** | 1.515773 | -0.278429 | 0.669981 | 1.0 | 0.0 | 0.0 | 0.0 | |

795 rows × 13 columns

MATCHING THE COLUMN ORDERS

In [104…
```
X_test_Final = X_test_Final[X_train_Final.columns]
```

In [105…
```
X_test_Final
```

Out[105…

| | Distance_km | Preparation_Time_min | Speed[Km/m] | Weather_Clear | Weather_Rainy | Weather_Windy | Traffic_Level_High | Tra |
|---|---|---|---|---|---|---|---|---|
| **0** | -1.341724 | -0.002430 | -0.902557 | 1.0 | 0.0 | 0.0 | 0.0 | |
| **1** | 0.215781 | 0.411568 | -0.305898 | 0.0 | 1.0 | 0.0 | 0.0 | |
| **2** | -0.938584 | 1.515564 | -0.874705 | 0.0 | 1.0 | 0.0 | 1.0 | |
| **3** | -1.224511 | 0.963566 | -0.928884 | 1.0 | 0.0 | 0.0 | 0.0 | |
| **4** | -0.933256 | -0.002430 | -0.708013 | 0.0 | 1.0 | 0.0 | 0.0 | |
| **…** | … | … | … | … | … | … | … | |
| **194** | 0.752117 | -0.830427 | 0.761835 | 0.0 | 1.0 | 0.0 | 0.0 | |
| **195** | -1.355931 | -0.554428 | -0.841917 | 0.0 | 0.0 | 1.0 | 0.0 | |
| **196** | 0.370288 | 1.653563 | -0.518019 | 0.0 | 0.0 | 1.0 | 0.0 | |
| **197** | -0.556756 | -0.554428 | -0.344169 | 0.0 | 0.0 | 0.0 | 0.0 | |
| **198** | -0.629570 | -0.968426 | -0.167856 | 1.0 | 0.0 | 0.0 | 1.0 | |

199 rows × 13 columns

APPLYING LINEAR REGRESSION MODEL

In [106…
```
Y_test.to_csv('Y_test_org')
```

In [107…
```
Y_test_org = pd.DataFrame(Y_test, columns=['Delivery_Time_min'])
Y_test_org
```

Out[107…

| | Delivery_Time_min |
|---|---|
| **925** | 28 |
| **529** | 57 |
| **571** | 63 |
| **660** | 44 |
| **932** | 34 |
| **...** | ... |
| **490** | 68 |
| **455** | 28 |
| **66** | 68 |
| **143** | 37 |
| **688** | 44 |

199 rows × 1 columns

In [108…

```
#rechecking columns from both the dataset

print("Training Features:", X_train_Final.columns)
print("Testing Features:", X_test_Final.columns)
```

```
Training Features: Index(['Distance_km', 'Preparation_Time_min', 'Speed[Km/m]', 'Weather_Clear',
       'Weather_Rainy', 'Weather_Windy', 'Traffic_Level_High',
       'Traffic_Level_Low', 'Time_of_Day_Afternoon', 'Time_of_Day_Evening',
       'Time_of_Day_Morning', 'Vehicle_Type_Bike', 'Vehicle_Type_Car'],
      dtype='object')
Testing Features: Index(['Distance_km', 'Preparation_Time_min', 'Speed[Km/m]', 'Weather_Clear',
       'Weather_Rainy', 'Weather_Windy', 'Traffic_Level_High',
       'Traffic_Level_Low', 'Time_of_Day_Afternoon', 'Time_of_Day_Evening',
       'Time_of_Day_Morning', 'Vehicle_Type_Bike', 'Vehicle_Type_Car'],
      dtype='object')
```

PREDICTION ON X_TEST DATASET

In [109…
```
Y_predict = model.predict(X_test_Final)
```

In [110…
```
Y_predict
```

Out[110…

```
array([[ 33.29989564],
       [ 67.45961385],
       [ 60.19400536],
       [ 41.55770475],
       [ 38.61419953],
       [ 36.65320822],
       [ 67.25643946],
       [ 79.6218329 ],
       [ 36.36037659],
       [ 28.10051262],
       [ 26.59273622],
       [ 58.75239451],
       [ 52.39425688],
       [ 52.98621232],
       [ 42.50390234],
       [ 40.90099186],
       [ 93.52657557],
       [ 73.87419794],
       [ 48.09290896],
       [ 63.90919862],
       [ 85.06131948],
       [ 19.03775582],
       [ 58.69177637],
       [ 19.90580575],
       [ 19.16235147],
       [ 44.4597105 ],
       [ 34.84551479],
       [ 50.7051786 ],
       [ 78.26122565],
       [ 83.69120392],
       [ 56.89615438],
       [ 51.36815856],
       [ 71.61422528],
       [ 65.51399821],
       [ 34.67437332],
       [ 86.516648  ],
       [ 49.63610333],
       [ 43.56133508],
       [102.35736686],
       [ 97.52731831],
       [ 65.38460358],
       [ 61.42971054],
```

```
       [ 60.79457932],
       [ 60.45869995],
       [ 94.15128582],
       [ 59.76253541],
       [ 94.72657255],
       [ 62.63466577],
       [ 80.63020253],
       [ 48.67315053],
       [ 45.86755842],
       [ 71.83897189],
       [ 58.17529292],
       [ 53.34255113],
       [ 75.41310686],
       [ 41.95405363],
       [ 26.19144805],
       [ 90.76807028],
       [ 37.13879879],
       [ 24.06533393],
       [ 65.32042937],
       [ 61.10708157],
       [ 52.93278473],
       [ 81.93492857],
       [ 40.19730731],
       [ 64.49201965],
       [ 63.0991401 ],
       [ 53.27718568],
       [ 33.59689684],
       [ 53.77909821],
       [ 57.01371818],
       [ 46.53057526],
       [ 72.05374529],
       [ 28.39972644],
       [ 87.19851086],
       [ 32.90605132],
       [ 47.70055753],
       [ 69.03785214],
       [ 50.62893232],
       [ 48.6861068 ],
       [ 65.68701862],
       [ 24.16392847],
       [ 47.11853624],
       [ 21.31124241],
```

```
       [ 39.50241162],
       [ 76.76681808],
       [ 71.4303669 ],
       [ 52.41144681],
       [ 60.41635451],
       [ 34.91096671],
       [ 72.19465979],
       [ 73.78660792],
       [ 80.95735302],
       [ 16.73026457],
       [ 27.70989964],
       [ 58.39527814],
       [ 83.56795348],
       [ 20.66694563],
       [ 37.74307846],
       [ 47.4247735 ],
       [ 67.940309  ],
       [ 65.36524668],
       [ 40.15854225],
       [ 35.79278606],
       [ 73.17195623],
       [ 46.15933304],
       [ 58.30148195],
       [ 35.20186383],
       [ 83.0612123 ],
       [ 45.47949589],
       [ 35.35393665],
       [ 96.00181661],
       [ 28.18956119],
       [ 35.0950032 ],
       [ 54.12039798],
       [ 90.04389872],
       [ 79.21743089],
       [ 45.95884106],
       [ 34.91967793],
       [ 36.19599481],
       [ 37.24796871],
       [ 37.28937477],
       [ 54.69123953],
       [ 61.51563017],
       [ 50.31980891],
       [ 61.36280433],
```

```
       [ 40.0070615 ],
       [ 74.54606135],
       [ 46.94220588],
       [ 66.53559296],
       [ 46.1271695 ],
       [ 65.29941857],
       [ 92.98011452],
       [ 24.63768742],
       [ 16.82809341],
       [ 76.67585012],
       [ 67.551539  ],
       [ 34.14095997],
       [ 80.70736691],
       [ 38.49042428],
       [ 35.2223722 ],
       [ 71.67565047],
       [ 79.12860663],
       [ 83.00138979],
       [ 44.69058481],
       [ 51.81918544],
       [ 32.31016119],
       [ 63.07344284],
       [ 63.44145312],
       [ 52.20355381],
       [ 80.96777074],
       [ 58.19442198],
       [ 34.69726026],
       [ 72.25860275],
       [ 67.10177887],
       [ 51.79942658],
       [ 40.04009501],
       [ 74.96226285],
       [ 38.26200107],
       [ 95.73591935],
       [ 50.39804269],
       [ 28.57816835],
       [ 73.18936893],
       [ 29.34756692],
       [ 61.48918889],
       [ 74.02447252],
       [ 29.90337464],
       [ 46.97490408],
```

```
[ 28.49166373],
[ 32.16862875],
[ 45.78161791],
[ 29.98678384],
[ 14.29004876],
[ 18.85530949],
[ 50.71541962],
[ 54.81663621],
[ 45.98474303],
[ 74.84026611],
[ 25.57035765],
[ 72.76353361],
[ 48.1745701 ],
[ 25.91365216],
[ 56.63691944],
[ 65.83214331],
[ 62.10754682],
[ 49.25652169],
[ 53.49506512],
[ 93.50265388],
[ 68.12414965],
[ 69.59344875],
[ 41.62209306],
[ 18.02090305],
[ 38.77984721],
[ 60.63740446],
[ 65.14773904],
[ 31.3675983 ],
[ 68.043484  ],
[ 43.70279279],
[ 44.08664327]])
```

# CHECKING MODEL ACCURACY

In [111…
```
from sklearn.metrics import r2_score, mean_squared_error
```

In [112…
```
r2 = r2_score(Y_test_org, Y_predict)
```

In [115…
```
print("The R2 score is:", r2)
```

```
The R2 score is: 0.8415282752722824
```

In [116…

```
print("The model can successfully predict")
```

```
The model can successfully predict
```